



SEMI E39-0703

OBJECT SERVICES STANDARD: CONCEPTS, BEHAVIOR, AND SERVICES

This standard was technically approved by the Global Information & Control Committee and is the direct responsibility of the North American Information & Control Committee. Current edition approved by the North American Information & Control Committee on March 12, 2003. Initially available at www.semi.org May 2003; to be published July 2003. Originally published in 1995; previously published November 2002.

CONTENTS

1 Purpose

2 Scope

3 Referenced Standards

4 Terminology

4.1 Definitions

4.2.1 Requirements

4.2.2 Objects

4.2.3 Services

4.2.4 Data Type

5 Conventions

5.1 OMT Object Information Model

5.2 Object Attribute Representation

5.3 Service Message Representation

6 Background

7 Applicable Documents

8 Basic Concepts and Behavior

8.2 Object Attributes

8.2.3 Use of Text

8.2.4 Object Type

8.2.5 Object Identifier

8.2.6 Object Handle

8.3 Object Definition

9 Object Relationships

9.2 Object Hierarchy

9.3 Scope and Ownership

9.4 Multiple Inheritance Hierarchy

10 Additional Operations

10.2 Create and Delete

10.2.1 Create

10.2.2 Delete

10.3 Attachment

10.3.3 Attach

10.3.4 Attach Set Attributes

10.3.5 Detach

10.3.6 Reattach

10.3.7 Attach Supervised Object

10.3.8 Detach Supervised Object

11 Services and Scenarios

11.2 Scope

11.3 Filtering

11.4 Object Services Parameter Dictionary

11.5 Get and Set Attributes

11.6 GetType

11.7 GetAttrName

12 Additional Services

12.2 Object Services Parameter Dictionary

12.3 Create

12.4 Delete

12.5 Attach

12.6 AttachSetAttr

12.7 Detach

12.8 Reattach

12.9 AttachSupervisedObject

12.10 DetachSupervisedObject

12.11 ObjectAction

12.12 ObjectActionCompletion Notification

12.13 Get Service Names

12.14 Get Service Parameters

13 Applications

13.2 Get All of an Object' s Attributes

13.3 Determine All Objects of a Specific Type and with Specific Characteristics

13.4 Determine Specific Attributes of a Specific Object Instance

13.5 Determine Types of Subordinate Objects

13.6 Determine Names of Attributes of Subordinate Objects

14 Requirements for Compliance

14.2 Fundamental Requirements

14.3 Additional Capabilities

14.3.1 Filters

14.3.2 Owner Objects

14.3.3 Multiple Inheritance Hierarchy

APPENDIX 1

A1-1 Overview of Object Terminology

A1-2 Object Modeling Technique (OMT) Notation

A1-2.1 Basic Notation

A1-2.2 Associations

A1-3 Generalization and Inheritance

A1-3.1 Object Composition and Containment

LIST OF TABLES

Table 1 Top Object Attribute Definition

Table 2 Object Services

Table 3 Object Services Parameter Dictionary

Table 4 GetAttr Service

Table 5 SetAttr Service

Table 6 GetType Service

Table 7 GetAttrName Service

Table 8 Additional Object Services

Table 9 Additional Object Services Parameter Dictionary

Table 10 Create Service

Table 11 Delete Service

Table 12 Attach Service

Table 13 AttachSetAttr Service

Table 14 Detach Service

Table 15 Reattach Service

Table 16 AttachSupervisedObject Service

Table 17 DetachSupervisedObject Service

Table 18 ObjectAction Service

Table 19 ObjectActionCompletion Notification

Table 20 GetServiceNames Service

Table 21 GetServiceParameters

SEMI E39-0703

OBJECT SERVICES STANDARD: CONCEPTS, BEHAVIOR, AND SERVICES

This standard was technically approved by the Global Information & Control Committee and is the direct responsibility of the North American Information & Control Committee. Current edition approved by the North American Information and Control Committee on March 12, 2003. Initially available at www.semi.org May 2003; to be published July 2003. Originally published in 1995; previously published November 2002.

1 Purpose

1.1 The purpose of the Object Services Standard (OSS) is to provide general terminology, conventions, and notation for describing behavior and data in terms of objects and object attributes. In addition, it provides basic services for reading object attributes, setting their values, and for asking for an object's contents. This standard is intended to be referenced by other standards which define specific objects to reduce redundancy.

2 Scope

2.1 The scope of this standard is to provide concepts, behavior, and services common to a variety of public objects.

2.2 Object models are common to multiple standards. Object Services provide basic object-related definitions, and basic services for getting object attributes and setting attribute values, that can be used by all standards defining public objects. These services allow basic management of data based on objects.

2.3 The object services defined in this document may be included in the services provided by other standards. They may also be provided independently of such other standards.

NOTICE: This standard does not purport to address safety issues, if any, associated with its use. It is the responsibility of the users of this standard to establish appropriate safety and health practices and determine the applicability of regulatory or other limitations prior to use.

3 Referenced Standards

3.1 ISO Standard¹

ISO 9595 — Common Management Information Service (CMIS)

1 International Organization for Standardization, ISO Central Secretariat, 1, rue de Varembe, Case postale 56, CH-1211 Geneva 20, Switzerland. Telephone: 41.22.749.01.11; Fax: 41.22.733.34.30 Website: www.iso.ch

3.2 Other Standards

James Rumbaugh, Michael Blaha, William Premerlani, Frederick Eddy, William Lorensen, *Object-Oriented Modeling and Design*, Englewood Cliffs, New Jersey: Prentice-Hall, 1991.

NOTICE: Unless otherwise indicated, all documents cited shall be the latest published versions.

4 Terminology

4.1 The following basic definitions are provided in a logical order.

4.2 Definitions

4.2.1 Requirements

4.2.1.1 *fundamental compliance* — conformance to all fundamental requirements for an object or service resource.

4.2.1.2 *fundamental requirements* — the requirements for information and behavior that must be satisfied for compliance with a standard. Fundamental requirements apply to specific areas of application, objects, or services.

4.2.2 Objects

4.2.2.1 *object*² — an entity with a specific set of data and behaviors. Objects may be physical or conceptual.

4.2.2.2 *standardized object* — a object formally defined in SEMI standards and in compliance with the fundamental requirements of SEMI E39 (Object Services Standard: Concepts, Behavior, and Services).

4.2.2.3 *object model* — a static graphic model of objects to show structure — the identity of objects, their attributes and operations, and their relationships with one another.

4.2.2.4 *object type* — a formal classification of a group of similar objects. Synonym: *object class*. Examples: *equipment, wafer, carrier*.

2 The term “object” may refer to either an object class or an instance of a class.

4.2.2.5 *object instance* — an instance of an object type. An object type is like a template, while an object instance is the actual object. Example: an actual and specific optical stepper installed in a particular fab is an instance of the type “Optical Stepper.”

4.2.2.6 *object attribute (attribute)* — information concerning an object. Examples for object type “equipment”: manufacturer, model, serial number.

4.2.2.6.1 Attributes are classified in various ways, according to their visibility (public/private), optionality, and access:

- *fundamental attribute* — an attribute that is required for fundamental compliance with a standard service.
- *optional attribute* — an attribute that is required only in support of one or more optional standard services.
- *private attribute* — an attribute that is used strictly for internal purposes and is unknown (invisible) through public services.
- *public attribute* — an attribute that is known (visible) and whose current value is provided as a service to other entities upon request.
- *read-only attribute (RO)* — may not be changed through public services.
- *read/write attribute (RW)* — may be changed through public services.

4.2.2.7 *attribute name* — the formal name of the attribute that is used to identify it. The names (and data types) of public attributes are included with the object’s definition and are unique for that object.

4.2.2.8 *object identifier* — a set of one or more items of information, concerning a particular instance (instantiation) of an object of a given type, that together uniquely distinguish that instance from all other instances of that object within a defined scope. NOTE: An object may have more than one identifier. An identifier may be *simple* (consist of only one attribute) or *complex* (consist of more than one attribute). Example: The combination of equipment’s manufacturer, model, and serial number serve as an identifier that uniquely identifies a specific installation.

4.2.2.9 *object handle* — a numeric or binary identifier assigned by an application for internal use. NOTE: A handle may also be available as a *public attribute* but cannot generally be guaranteed to be either persistent or unique outside of the current relationship between service user and service provider. The persistence of the handle is specified by the standard that defines the object.

4.2.2.10 *operation* — a function performed by, or inherent to, an object. Example: for equipment, “run,” “stop,” “abort.”

4.2.2.11 *aggregation object* — an object that is composed (made up) of other objects. An aggregation may lose some degree of integrity if one of its components is missing.

4.2.2.12 *component object* — an object that is part of an aggregation.

4.2.2.13 *container object* — an object that is intended to hold other types of objects. The contents may or may not be ordered.

4.2.2.14 *contents* — an object that is in a container. Examples: a wafer in a cassette, a book in a library.

4.2.2.15 *owner object* — an object that is an aggregation, container, or supervisor of another object. The owner object is said to *own* the other object.

4.2.2.16 *owned object* — an object that is a component of, contained in, or supervised by, another object. The owned object is said to be *owned by* the other object.

4.2.2.16.1 An object may have multiple owners. For example, it may be a shared resource.

4.2.3 Services

4.2.3.1 *scope* — the specification of one or more objects that starts with a specific owner object and proceeds downward through a hierarchical sequence of “owns” relationships.

4.2.3.2 *service (or message service)* — represents a function offered to a user by a provider. A service consists of a sequence of service primitives, each described by a list of parameters. A service excludes definition of message structure and protocol.

4.2.3.2.1 A service may or may not be processed by the provider of the service. The invocation of some service may have to interact with other objects to fulfil its requirement completely; an exception will be raised if the service can not fulfill all its pre or post conditions. The object that provides the service may reject the service request or restrict the completion of the service to prevent an illegal action due to the equipment condition.

4.2.3.3 *notification service* — initiated by the service provider and sent to the service consumer/subscriber. No response is expected.³

4.2.3.3.1 Notifications consist of two service primitives — a message from the sender to the communications

³ This does not preclude the implementation of a response message in the message protocol.

facility and an indication to the receiver from the communications facility.

4.2.3.4 *request service* — initiated by the service consumer. Requests ask for data or for an activity (operation) from the provider. Requests expect a specific response message.

4.2.3.4.1 A request consists of a message that requires a response from the receiver. The primitives for a request are the same as those of the notification, while the response defines additional primitives called the *response* and the *confirmation*.

4.2.3.5 *service resource* — a logical group of one or more services within a specific area of functionality.

4.2.4 Data Type

4.2.4.1 *form* — type of data: positive integer, unsigned integer, integer, enumerated, boolean, text, formatted text, structure, list, ordered list.

4.2.4.2 *positive integer* — may take the value of any positive whole number. Messaging protocol may impose a limit on the range of possible values.

4.2.4.3 *unsigned integer* — may take the value of any positive integer or zero. Messaging protocol may impose a limit on the range of possible values.

4.2.4.4 *integer* — may take on the value of any negative or unsigned integer. Messaging protocol may impose a limit on the range of possible values.

4.2.4.5 *floating point* — may take on any single (real) numeric value, positive or negative. Messaging protocol may impose a limit on the range of possible values.

4.2.4.6 *enumerated* — may take on one of a limited set of possible values. These values may be given logical names, but they may be represented by any single-item data type.

4.2.4.7 *boolean* — may take on one of two possible values, equating to TRUE or FALSE.

4.2.4.8 *text* — A text string. Messaging protocol may impose restrictions, such as length or ASCII representation.

4.2.4.9 *formatted text* — a text string with an imposed format. This could be by position, by use of special characters, or both.

4.2.4.10 *structure* — a complex structure consisting of a specific set of items, of possibly mixed data types, in a specified arrangement.

4.2.4.11 *list* — a set of one or more items that are all of the same form (one of the above forms).

4.2.4.12 *ordered list* — a list for which the order in which items appear is significant.

5 Conventions

- Defined terms are presented in boldface when introduced for the first time.
- Formally reserved text strings, such as attribute names, are underlined.

5.1 *OMT Object Information Model* — The object models are presented using the Object Modeling Technique (OMT) developed by Rumbaugh, James, et al, in *Object-Oriented Modeling and Design*, Prentice Hall, Englewood Cliffs, NJ, c1991.⁴

5.1.1 Overviews of this notation are provided in the Appendix. A brief discussion of terminology is also provided in the Appendix.

5.2 *Object Attribute Representation* — The object information models for standardized objects will be supported by an attribute definition table with the following column headings:

<i>Attribute Name</i>	<i>Definition</i>	<i>Access</i>	<i>Rqmt</i>	<i>Form</i>
The formal text name of the attribute	Description of the information contained	RO or RW	Y or N	(See Section 4.2.4.)

5.2.1 The Access column uses RO (Read Only) or RW (Read and Write) to indicate the access that users of the service have to the attribute.

5.2.2 A “Y” or “N” in the Requirement (Req) column indicates if this attribute must be supported in order to meet fundamental compliance for the service.

5.2.3 The Form column is used to indicate the format of the attribute. (See Section 4.2.4 for definitions.)

5.3 Service Message Representation

5.3.1 *Service Resource Definition* — A service resource definition table defines the specific set of messages for a given service group, as shown in the following table:

<i>Message Service Name</i>	<i>Type</i>	<i>Description</i>
Message name	N or R	The intent of the service.

5.3.1.1 Type can be either N = Notification or R = Request.

5.3.1.2 Notification type messages are initiated by the service provider, and the provider does not expect to get a response from the consumer/subscriber.

4 For a full description of the Object Modeling Technique, see *Object-Oriented Modeling and Design*, Rumbaugh, Blaha, Premerlani, Eddy and Lorenson, Prentice Hall, 1991.

5.3.1.3 Request messages are initiated by a service consumer or subscriber. Request messages ask for data or an activity from the provider. Request messages expect a specific response message (no presumption on the message content).

5.3.2 *Service Parameter Dictionary* — A service parameter dictionary table defines the parameters for one or more services, as shown in the following table:

Parameter	Form	Description
Parameter X	Data type	A parameter called X is B in A.

5.3.2.1 A row is provided in the table for each parameter of the service. The first column contains the name of the parameter. This is followed by columns describing the form and contents of the corresponding primitive.

5.3.2.2 The Form column is used to indicate the type of data contained in a parameter. (See Section 4.2.4 for definitions.)

5.3.2.3 The Description column in the Service Parameter Dictionary table describes the meaning of the parameter, the values it can assume, and any interrelationships with other parameters.

5.3.2.4 To prevent the definition of numerous parameters named “XxxList,” this document adopts the convention of referring to the list as “(List of) Xxx.” In this case, the definition of the variable Xxx will be given, not of the list. The term “list” indicates a collection (or set) of zero or more items of the same data type. Where a list is used in both the request and the response, the list order in the request is retained in the response. A list must contain at least one element unless zero elements are specifically allowed.

5.3.3 *Service Message Definition* — A service message definition table defines the parameters used in a service, as shown in the following table:

Parameter	Req/Ind	Rsp/Conf	Description
Parameter X	(see below)	(see below)	A description of the service.

5.3.3.1 The columns labeled Req/Ind and Rsp/Conf link the parameters to the direction of the message. The message sent by the initiator is called the “Request.” The receiver terms this message the “Indication” or the request. The receiver may then send a “Response,” which the original sender terms the “Confirmation.”

5.3.3.2 The following codes appear in the Req/Ind and Rsp/Conf columns and are used in the definition of the

parameters (e.g., how each parameter is used in each direction):

- “M” — *Mandatory Parameter* — must be given a valid value.
- “C” — *Conditional Parameter* — may be defined in some circumstances and undefined in others. Whether a value is given may be completely optional or may depend on the values of other parameters.
- “U” — *User-Defined Parameter*
- “-” — The parameter is not used.
- “=” — (for response only) Indicates that the value of this parameter in the response must match that in the primary (if defined).

6 Background

6.1 During the development of proposals for several other service standards, it was discovered that they shared a common need for terminology and services related to objects. Originally each proposal included a service-specific message for getting the values of an object’s attributes, and some also provided a service-specific message for setting these values.

6.1.1 The decision was made to eliminate this redundancy through provision of a standard to provide those definitions and services used by the other separate service standards.

7 Applicable Documents

SEMI Book of Standards, Equipment Automation/Software Volumes 1 and 2⁵

ISO/TR 8509:1987,⁶ Information Processing Systems, Open Systems Interconnection — Service Conventions

William Stallings, Networking Standards, *A Guide to OSI, ISDN, LAN, and MAN Standards*, Addison-Wesley Publishing Company

8 Basic Concepts and Behavior

8.1 This section defines the concepts and behavior that are common to public objects.

8.1.1 Object-oriented analysis is a widely accepted tool, both for applications that are implemented using

⁵ This set of documents may be obtained from SEMI, 3081 Zanker Road, San Jose, CA 95134.

⁶ International Organization for Standardization, ISO Central Secretariat, 1, rue de Varembe, Case postale 56, CH-1211 Geneva 20, Switzerland. Telephone: 41.22.749.01.11; Fax: 41.22.733.34.30
Website: www.iso.ch

object-oriented technologies and those using traditional technologies. Object information models provide a powerful method for describing relationships that can be intuitively understood.

8.1.2 For those unfamiliar with object-oriented terminology, a brief discussion of terms is provided in the Appendix.

8.2 *Object Attributes* — An *object attribute* is a data value that is held by all instances of a given object type. For communication purposes, public attributes are assigned a logical *name* that is unique for that object. An object's attributes may be individually referenced either by an enumerated numeric value or by a reserved *attribute name* that conforms to the use of text defined in Section 8.1.1.

8.2.1 An attribute that can be requested or referenced through formal public *services* is a *public attribute*. Other attributes may exist in specific implementations but are invisible through public services and are called *private attributes*.

8.2.2 Specific attributes may be designated as *fundamental* for a specific object. These shall be supported by all implementations that use or reference the specified object. Attributes that are not fundamental are *optional*.

8.2.3 *Use of Text* — Values defined as text, when given in ASCII, are subject to certain restrictions. This includes object attribute values, attribute names, and service parameters.

8.2.3.1 Text in ASCII is restricted to the characters between 20₁₆ and 7E₁₆, excluding the question mark “?”, the asterisk “*”, and the tilde “~”.

8.2.3.2 The question mark and asterisk are reserved for use as “wild characters” in filters and searches, while the tilde is reserved to allow systems that cannot use spaces to convert spaces to tildes for internal use.

8.2.3.3 Text used in specific contexts may have additional restrictions.

8.2.3.4 Unless otherwise stated, case is not significant for purposes of comparison. However, case is used to improve readability and should be preserved whenever possible.

8.2.4 *Object Type* — An object always knows its type. For this reason, the attribute for object type, ObjType, is required for all public objects. ObjType is a text string containing the formal classification of an object. The text string shall conform to the convention defined in Section 8.1.1, with the additional restriction that the “greater than” symbol “>” and the colon “:” are also excluded. The text string shall not start or end with a space.

8.2.4.1 Object types (the values for ObjType) for standardized objects are reserved. Types reserved are specific to individual standards.

8.2.5 *Object Identifier* — Every instance of an object shall have one or more attributes that together uniquely distinguish that instance from all other instances of objects of the same object type. An object may have more than one identifier. The identifier(s) are defined as part of the object definition.

8.2.5.1 ObjID is a text-based fundamental attribute of all public objects and provides an attribute of a known form to serve as an identifier for an object of any type. The text string shall conform with the convention defined in Section 8.1.1 and with the additional restriction that the “greater than” symbol “>” and the colon “:” are also excluded. The text string shall not start or end with a space.

8.2.5.2 From the point of view of objects such as managers, aggregates, and containers that have responsibilities for other objects (see Sections 9.1 and 9.2) the combination of ObjType and ObjID for these other objects shall be unique. For example, if several process chambers have a single vacuum pump, each process chamber may have direct access to at most one ObjType:ObjID combination of “Pump:Vacuum”. However, the combination is not required to be unique across the different chambers. The host differentiates between the several vacuum pumps by their belonging to different aggregations.

8.2.5.3 The object definition for each object shall specify the particular identifier used for general access. For objects that normally use an identifier with a numeric value as an identifier (such as “handle,” described below), the value of ObjID is a numeric text string set to the value of that attribute.

8.2.5.4 An object may also have other identifiers in addition to ObjID.

8.2.6 *Object Handle* — An object's *handle* is an attribute with a numeric value that is assigned by the application that created the object. The handle may be used for the object's identifier. It is generally intended for local use and may or may not be defined as a public attribute in the object's formal definition.

8.2.6.1 Where used, the handle may be guaranteed to be unique only within a specific context and within a single association between service-provider and service-user for a specific service resource. The handle may or may not be persistent beyond a certain context within that association or beyond it. Persistence of the handle is specified as part of the definition of the object and is beyond the scope of OSS.

8.3 *Object Definition* — An object definition includes an attribute definition table described in Section 5.3.

8.3.1 The object that is the super-type of all public objects is called the *top object*. The attributes and operations of the top object, shown in Figure 1, are fundamental requirements for all public objects. That is, all objects shall respond to the object attributes ObjType and text strings, and all public objects shall recognize and respond to the operations *get attributes* and *set attributes*.

Top
ObjType: text ObjID: text
get attributes set attributes

Figure 1
Top Object

8.3.2 Formal definition of an object shall include a table defining the object' s public attributes. The Object Attribute Definition table for the top object is given in Table 1 below. It contains the two fundamental attributes required of all public objects.

Table 1 Top Object Attribute Definition

Name	Definition	Access	Req	Form
<u>ObjType</u>	The object type.	RO	Y	Text
<u>ObjID</u>	The object' s identifier.	RO	Y	Text

8.3.3 Access refers to the ability to read and write the value of the attribute through OSS services. An object' s type and identifier may not be changed through OSS services. However, it may be possible for the service user to assign a value for the object' s identifier at the time the object (instance) is created. This depends on the object and the services provided.

9 Object Relationships

9.1 This section addresses relationships between objects that affect communications, either by restricting communications or through information concerning relationships.

9.2 *Object Hierarchy* — Certain objects may be specified as *aggregation objects* or as *container objects* when they are defined.

9.2.1 An aggregation object is composed of other objects called the *components* of the aggregation. This

is illustrated in Figure 2. Components may be of one or more different types, or they may be of the same type.

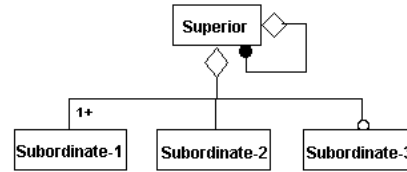


Figure 2
Aggregation with Two Component Types

9.2.2 Container objects *contain* other objects, of the same or different types, but are not made up of them.

9.2.3 Figure 3 shows two object types that are associated with a relationship called “contains.” The solid circle indicates that Type-1 may contain zero or more objects of Type-2 type. An example of a common container type is a file directory.

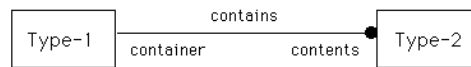


Figure 3
Container and Contents

9.2.4 An aggregation loses some degree of integrity if certain of its components are missing, while a container retains full integrity even if it has no contents. An automobile is an aggregation of many components, some of which are themselves aggregations. An automobile may also have contents (driver, passengers, belongings) which are removable and not considered as components. Other examples of container objects include lists, dictionaries, and libraries. A container that has no contents is said to be *empty*.

9.2.5 The aggregation or container is called the *superior object* and its components or contents are called *subordinate objects*.

9.2.6 In addition to the relationships of “is composed of” (roles: aggregate/component) and “contains” (roles: container/contents), the other hierarchical relationship that occurs naturally in factories and in control systems is that of “*supervises*” (roles: supervisor/supervised). This denotes a control relationship where the supervised object accepts part or all of its directions from the supervisor. The supervisor, in turn, has responsibilities that it delegates to the supervised. The supervisor typically will also have a relationship of “*is composed of*” or “*contains*” with its supervised subordinates.

9.3 *Scope and Ownership* — The hierarchical relationships of aggregations, containers, and supervisors may at times be required for pointing to a specific object. These relationships may be shown with a multi-level tree structure, as shown in Figure 4. Subordinate objects at one level may be the superior objects at the next lower level. The superior object at the highest level is called the *root* of the tree.

NOTE 1: Figure 4 is not drawn in OMT notation.

9.3.1 An *owner object* is an object that is an aggregate, container, or supervisor. An *owned object* is an object that is a component of, contained in, or supervised by, an owner object.

9.3.2 *Scope*⁷ is the concept and method of pointing to a specific owned object through the use of sequence of hierarchical relationships. Scope allows any particular object (type or instance) within such a hierarchical tree to be fully specified by providing a unique path down this tree using a concatenation of object types and identifiers.

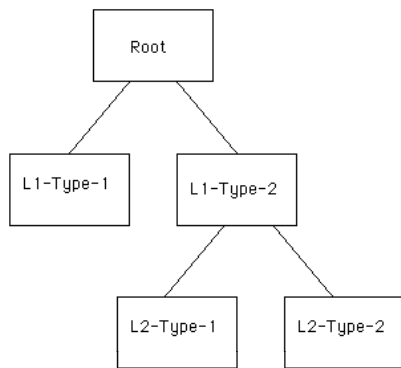


Figure 4
Example of Owner Hierarchy

9.3.3 This path is used to create an *object specifier* that is represented as a sequence of concatenated text strings of the form “type:id>”, which may be repeated as necessary to form the complete path. The character “>” is used to terminate the object type field, and the colon character “:” is used to terminate the identifier field. The object specifier uniquely identifies an object within the entire domain of objects and is able to extend the domain within which a search for an object would otherwise occur.

9.3.4 Formally, an object specifier is a formatted text string of the form:

“type₁:id₁>...type_n:id_n>”

where “type_i” and “id_i” represent the object type and object identifier, respectively, of the *i*th object instance in the sequence, and where the *i*th object is owned by the (*i*-1)th object and is the owner of the (*i*+ 1)th object.

9.3.5 Figure 5 shows an example of a typical set of hierarchical relationships in the factory. An application of scope might be through Cell AB, Cluster BB, Process Module (PM) CB, to Device DB. In this example, the object specifier for Device DB would be:

“FactoryHost:Hilda>Cell:AB>Cluster:BB>PM:CB>Device:DB>”

9.3.6 It is generally not necessary to start with the root object. It is sufficient to start with an object that is owned by the communications partner. However, it is invalid to omit a level in the hierarchy that is between the starting object and the final object in the path.

9.3.7 Some systems may be able to guarantee the uniqueness of object identifiers for their owned objects. For example, equipment may be able to guarantee that all of its owned objects have unique identifiers regardless of type. In this case, an object’s identifier is sufficient to point to a specific object instance. Object type may be omitted without ambiguity, as it may be obtained as an attribute of the specified object. The form of the path then becomes “ id₁>id₂>...id_n> ”. In the previous example, this might be “ AB>BB>CB>DB> ”.

9.3.8 Object type may also be omitted in applications where it may be inferred without ambiguity within a particular context. A service that is dedicated to a specific object type X, for example, that is always owned only by another specific object type Y, may specify usage for object type X. Since object type may then be inferred from the context, object types X and Y need not be included in a specifier for an object of type X.

9.3.9 The terminator of the final identifier is optional. This allows a single object identifier to be used as a simple object specifier. As a result, however, it may be necessary to provide string delimiters to prevent premature termination of string parsing in the event an identifier contains one or more embedded spaces.

9.3.10 Scope can be applied to any set of hierarchical relationships, given a minimum set of requirements for public objects:

- an object always knows its own type (ObjType).
- an object always knows its own identifier(s) (including ObjID).
- an owner object knows the types of objects that are its components and/or contents, or that it otherwise

⁷ Scoping is used in material ISO 9595 (CMIS).

supervises, and is able to determine their identifiers (see GetType, Section 11.6),

- an owner object is able to determine the types and identifiers of its owned objects.

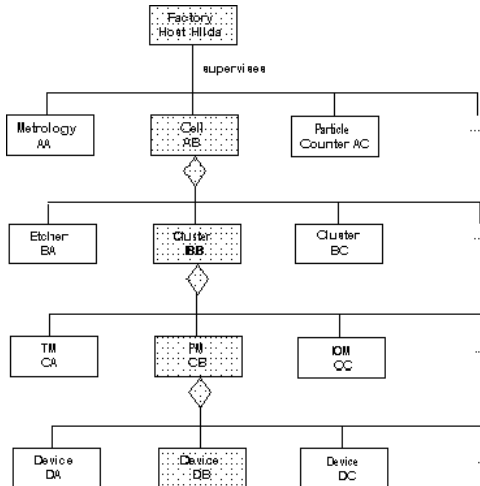


Figure 5
Hierarchical Relationships in the Factory

9.4 Multiple Inheritance Hierarchy — Multiple hierarchical inheritance may be required as an optional capability for pointing to a specific object that provides the same services from the inherited objects. Usually, inherited services are implemented by the domain object such that specifying “overrides” of attributes or services from inherited objects is not required. Sometimes the capability to override specified attributes and services from inherited object is necessary to resolve duplicated attribute names or services or to allow duplicated names in user extended objects.

9.4.1 If Andy an object TypeA inherits from object TypeB and TypeC and a service requestor is focusing services or attributes of the object TypeB rather than those with the same name in the object TypeC, the following text string form could specify the focused part of the object Andy:

TypeA@TypeB: Andy

9.4.2 If any names of service and/or attribute that TypeB provides are the same as those provided by TypeA or TypeC, then, the object Andy will have to be specified with the complete explanation from the supplier as how the object handles the exception and the differences from the original attribute or services. So, the above expression is the same as “TypeA:Andy” without notes to describe difference.

10 Additional Operations

10.1 This section defines operations that are common to many types of public objects but are not required for all types of public objects. These operations are not required for OSS compliance. However, they may be required for support of specific object types defined by other standards that use OSS services.

10.2 *Create and Delete* — Creation and deletion are optional services for object lifecycle management. Object definitions may specify conditions under which the services are supported. Objects may be created and/or deleted in other ways. For example, the creation and deletion of certain types of persistent objects may be outside of the scope of OSS services, while transient objects may be created and/or deleted automatically by their owners as the result of other activities.

10.2.1 *Create* — The create operation creates an instance of an object type.

10.2.1.1 Initial settings for one or more attributes may be specified by the service user. Initial settings may be required, prohibited, or optional, depending upon the specific object type. The ability to set attribute values through the create operation shall be clearly identified as part of the object definition for those objects supporting the create operation. For example, the ability to set the value of the object identifier ObjID may be required, optional, or prohibited, depending upon the type of object.

10.2.1.2 The object definition may also specify attributes of the new object instantiation that are returned to the service user by the service provider.

10.2.1.3 A request to create an object is invalid if it does not provide all required attribute settings, if it attempts to set prohibited attributes, or if it provides values invalid for a given attribute.

10.2.1.4 The definitions of certain objects may specify that the delete service only be provided to, or under the authority of, the service user that requested the original create operation. In this case, a private attribute (that is, an attribute not accessible through services defined in Section 11) named Delete Token shall return a unique integer as an attribute of the new object. This attribute is then used in a subsequent request to delete the object. The service user may delegate this token at its own discretion.

10.2.1.5 The create operation is invoked by the service Create sent to the owner that is to instantiate the specified object. In order for the Create service to be accepted if no owner for the object to be created is defined in the target entity or if it is physically outside of the domain of the entity; the object specifier shall define the owner as “ ” (an empty string). If the entity

is just the equipment rather than a component of the equipment; the owner may be defined as “Equipment” even if no such object is defined.

10.2.2 Delete — The delete operation is the inverse of the create operation. Individual object types may restrict deletion based on well-defined set of criteria. For example, the definition of a type of container may specify that a container may only be deleted when it is empty.

10.2.2.1 If an object to be deleted requires the authority of the service user that invoked the original create operation, then the Delete Token attribute of the target object shall be provided by the user of the delete service.

10.2.2.2 A request to delete an object is invalid if it does not provide all required attribute settings or if it provides values invalid for a given attribute.

10.2.2.3 The delete operation is invoked by the service Delete sent to the object to be deleted.

10.3 Attachment — When an object receives a remote request for an operation, it may be unable to identify the requestor. At the same time, it may be necessary for an object to determine if certain messages were sent by its supervisor. This conflict is resolved by formalizing the relationship between a supervised object and its supervisor.

10.3.1 Attachment is a dynamic behavioral binding between two objects in a hierarchical relationship where it is important that the attached (supervised) object be able to differentiate certain service requests made by its supervisor from other requests. An attached object has exactly one supervisor, as illustrated in Figure 6.

10.3.2 An attached object may be detached from its current supervisor, or it may be reattached to a different supervisor. The latter operation is used when the previous supervisor has become damaged and is not intended as a general method for moving the object’s

attachment, as the former supervisor is not notified of the change.

10.3.3 Attach — When sent to an unattached object, the attach operation creates a logical connection between the object and a supervisor. The object that is being attached creates a unique, private, non-zero numeric value, called an attach token. The attach token is treated as an attribute named AttachToken. It is a private attribute of the attached object, not visible through basic OSS services (see Section 11).

10.3.3.1 When an object is first created, it is unattached, and the value of its attach token is set to a default value of zero. An object is attached to another object as a formal relationship. The operation of attach generates the attach token, which is returned to the service user. The supervisor uses this token in subsequent critical service requests to identify itself to the attached object.

10.3.3.2 The attach operation also may accept settings for one or more attributes of the attached object, as defined for the attached object type. This allows the supervisor to provide values for attributes that are otherwise regarded as read-only values and cannot be set through the SetAttr service defined in Section 11. An example of an attribute value that it is often desirable to access in this way is the object specifier of the supervisor.

10.3.3.3 Requests to attach an object that attempt to set attributes shall be denied unless the attributes are among those accessible to the supervisor and all specified settings represent valid values of those attributes. The object type and identifier shall not be changed by the supervisor.

10.3.3.4 An attached object is owned by its supervisor.

10.3.3.5 The attach operation is invoked by the message service Attach by an intended supervisor. Requests to attach an already attached object shall be denied.

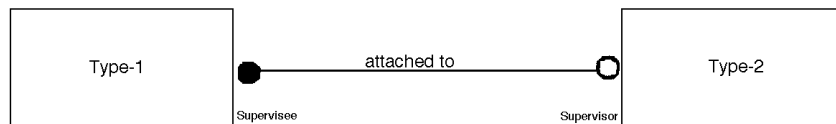


Figure 6
The Attachment Relationship

10.3.4 Attach Set Attributes — A supervisor may need the ability to set certain read-only attributes of one of its attached objects subsequent to the attach operation. The set attached attributes operation provides this capability to a supervisor of attached objects. The supervisor identifies itself by using the attach token for that object.

10.3.4.1 The definition of the attached object type may restrict the attributes that may be set by this operation. The attach set attributes operation shall be denied if the attributes or settings are invalid for this operation.

10.3.4.2 The attach set attributes operation is invoked by the message service AttachSetAttr sent by the object's supervisor and is otherwise invalid.

10.3.5 Detach — The detach operation breaks the logical connection between an attached object and its supervisor. The attach token is reset to zero, and the object becomes unattached.

10.3.5.1 The detach operation is invoked by the message service Detach sent by the object's supervisor and is otherwise invalid.

10.3.6 Reattach — The reattach operation is similar to the attach operation, except that it is sent only to an attached object to change the logical connection from the attached object to a new supervisor. This operation is used to replace a damaged supervisor. Any existing connections to the old supervisor shall be closed. Note the reattach operation is vulnerable to misuse. Applications supporting the reattach operation may have additional requirements, such as notification sent to the previous supervisor, to provide a trace of misuse.

10.3.6.1 The reattach operation provides a unique attach token that has not been previously used by the attached object. This effectively disables its previous supervisor.

10.3.6.2 An attempt to reattach an object shall be denied unless the attributes and settings are valid for the operation.

10.3.6.3 The reattach operation is invoked with the message service Reattach sent to an attached object by the object's new supervisor and is otherwise invalid.

10.3.7 Attach Supervised Object — The attach supervised object operation is invoked by a service user to request a supervisor to attach to itself a valid target object. If the supervised object does not already exist or is not a valid target type for the supervisor, then the attached supervised object operation shall fail. The results of the operation are returned to the service user.

10.3.7.1 The attach supervised object operation is invoked with the service AttachSupervisedObject.

10.3.8 Detach Supervised Object — The detach supervised object operation is invoked by a service user to request a supervisor to detach from itself an attached object. The supervisor sends a detach request to the specified attached object. Regardless of any response from the specified supervised object, the supervisor shall consider it thereafter as unattached. This is required to allow a damaged attached object to be removed. This operation is successful except when the target object is not already attached to the supervisor.

10.3.8.1 The detach supervised object operation is invoked with the service DetachSupervisedObject.

11 Services and Scenarios

11.1 The services defined by OSS are contained in Table 2.

Table 2 Object Services

<i>Message Service Name</i>	<i>Type</i>	<i>Description</i>
GetAttr	R	Get values of specified attributes based on attribute filters.
SetAttr	R	Set values of specified attributes based on attribute filters.
GetAttrName	R	Get names of attributes of objects owned by the service provider.
GetType	R	Get types for objects owned by the service provider.

11.2 Scope — Scope is supported through the object specifier parameter. The object specifier provides a method for pointing to objects owned by the service provider that are otherwise not accessible to the service user. That is, the service provider may not provide direct services for, or to, that object. This can occur when an object for which the service provider offers services is an owner object, such as an aggregate or file directory, as described in Section 9. In some cases, the owned object may itself be remote from the service provider. For example, the owned object may be provided by a separate system.

11.2.1 The service provider may be an owner object with managerial responsibilities, and therefore it may prohibit direct access to one of its component or contained objects. In this case, requests concerning an owned object from a service consumer shall be directed to the owner object instead.

11.2.2 Object services use the *object specifier* for the owner of the target object(s). The construction of the object specifier is defined in Section 9.

11.2.3 Object type may be omitted from the object specifier where it may be otherwise determined without ambiguity. However, inclusion of type is always valid.

11.3 *Filtering* — An *attribute filter* is an optional set of one or more *attribute qualifications*. An *attribute qualification* is a boolean expression that makes a statement about the presence or values of attributes in a target object. It identifies an attribute, a *qualifying value* of that attribute, and a *qualifying relationship* that the value has to the target attribute. The attribute filter is a boolean that consists of the expression formed by an AND of the set of qualifications.

11.3.1 If the attribute is of text form, the qualifying value may be used as a *mask* with the embedded *wild characters* “?” (question mark) and “*” (asterisk). The character “?” may be used within the mask to represent “any single character” and may be repeated. The string “?????” represents any text string with a length of five characters.

11.3.2 The character “*” may be used in the mask to represent a variable-length string, including a null string. The string “*x” represents a string of any length that ends in “x”; the string “x*” represents any string that begins with “x”.

11.3.3 When the character “*” is used by itself as the string “*”, however, it represents any string of any non-

zero length. It may also be repeated within a string, as in “*x*y*”, which represents any string with embedded characters “x” and “y”, such as “abxaby”, “x_y”, and “xy”.

11.3.4 The comparison for text characters shall be case insensitive.

11.3.5 The qualifying value specifies a qualifying relationship, “R”, between the value specified in the attribute qualification and the matching attribute of object instances. The qualifying value for the attribute is compared with the attribute of an instance of the target object to test the relationship.

11.3.6 If the qualifying value has the relationship “R” to the attribute of the target object, then the instance *qualifies* for the relationship and is included in the set of objects for which requested attributes are returned. For example, if the attribute name is “Length,” qualifying value is the number 5, and the qualifying relationship is “is less than,” then the Length attributes of all instances of the target object are tested. All instances with a “Length” attribute greater than 5 or equal to 5 qualify.

11.4 *Object Services Parameter Dictionary* — Table 3 defines all of the parameters, including the elements of complex parameters, used in object services.

Table 3 Object Services Parameter Dictionary

<i>Parameter</i>	<i>Definition</i>	<i>Form</i>
AttrData	The value of an attribute.	Varies with attribute. The form of an attribute' s value is specified as part of the definition of an object type.
AttrFilter	Attribute filter.	Structure composed of AttrName, AttrData, AttrReIn.
AttrName	The attribute' s name.	Text. Varies with object type.
AttrReIn	Qualifying relationship between the qualifying value and the matching attribute of object instances.	Enumerated: Equal To Not Equal To Less Than Less Than or Equal To Greater Than Greater Than or Equal To Present (specified attribute is present) Absent (specified attribute is absent) Contained (The qualifying value is in (i.e., equal to a member of) the set of the attribute's values.) Not Contained (If omitted, the relationship " Equal to" is assumed.)
AttrSetting	The name and value of an attribute.	Structure composed of AttrName and AttrData.

<i>Parameter</i>	<i>Definition</i>	<i>Form</i>
ErrorCode	Contains the code for the specific error found.	Enumerated: Unknown Object in Object Specifier Unknown Target Object Type Unknown Object Instance Unknown Attribute Name Read-Only Attribute — Access Denied
ErrorText	Text in support of the error code to provide additional information.	Text.
ObjAck	Acknowledgement code.	Unsigned integer. Possible values: Successful completion of request Error
ObjectAction	The name of an action to be performed on, or by, an object.	Text. Enumerated per individual object.
ObjectAction Acknowledgement	Status of action request.	Enumerated: Action was performed successfully. Action does not exist. Cannot perform now – try later. Action will be performed and notification sent later. Action is prohibited. Target object unknown.
ObjectActionParameter Request	Structure composed of a name and a value for a given object parameter used in the request.	Structure of ServiceParameterName and ServiceParameterValue.
ObjectActionParameter Result	Structure composed of a name and a value for given object parameter used in the result.	Structure of ServiceParameterName and ServiceParameterValue.
ObjectActionReportStatus	Status of receipt of an ObjectActionComplete-Notification.	Binary.
ObjectActionStatus	Result of the action request.	Structure of ObjectActionAcknowledgement and Status.
ObjectLinkID	Set to non-zero if and only if additional completion reports will be sent.	Unsigned integer.
ObjectServiceList	Specified the services supported by a given object type.	Structure composed of ObjType, ServiceName.
ObjID	Object identifier (ObjID).	Varies with object type.
ObjSetting	Attribute values for an object instance.	Structure composed of ObjID and (List of) AttrSetting.
ObjSpec	The object specifier, used to specify the owner of the target object.	Formatted text.
ObjStatus	Information concerning the success or failure of the operation.	Structure consisting of ObjAck and Status.
ObjType	Object type.	Object types are text strings reserved by individual standards. Object Services do not reserve object types.
OperationID	Identifies a specific request.	Unsigned Integer.
ServiceName	The name of a service or action to be performed on, or by, an object.	Text.
ServiceParameterDef	A structure of a service name and a list of the parameter	A structure of ServiceName and associated list of ServiceParameterNames.

Parameter	Definition	Form
	names that are supported by an object type.	
ServiceParameterName	Parameter name.	Text.
ServiceParameterValue	Parameter value.	Any. Varies depending on the parameter.
Status	Error information.	(List of) Structure composed of ErrorCode and ErrorText.

11.5 *Get and Set Attributes* — The GetAttr and SetAttr services are provided for reading and setting values of an object's attributes. They address attribute values of instances of objects. Figures 7 and 8 show the message flow for the GetAttr service.

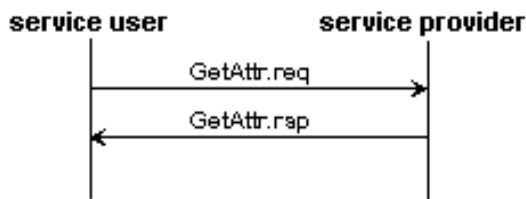


Figure 7
GetAttr Message Flow

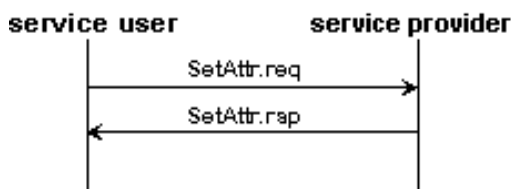


Figure 8
SetAttr Message Flow

11.5.1 GetAttr and SetAttr allow access to attributes of objects that are owned by another object and are capable of addressing multiple levels of ownership (object hierarchy).

11.5.1.1 For example, GetAttr may be used as a “get directory” request to find the identifiers of all instances of one or more owned objects.

11.5.2 GetAttr and SetAttr may be used for objects that use a complex identifier, such as recipes with separate attributes for name and version.

11.5.3 GetAttr and SetAttr allow searching for those instances of an object that satisfy certain conditions through the specification of one or more qualifiers or filters. Only attributes of those instances of the desired object type that satisfy the conditions defined in the filters are returned by the service provider.

11.5.3.1 The filter is familiar to users of file services as a convenient way to obtain a partial directory of files, such as files with a date later than a specified date. This

is particularly useful when the list of contents may be quite long.

11.5.4 The attribute value (all or a part of the list of ObjSetting) returned by the GetAttr service may be invalid if the acknowledge code of ObjStatus indicates a failure. In that case; the value(s) requested by the invocation of GetAttr service may be unknown, invalid or unavailable by any reason (e.g. corresponding hardware status, lack of privilege, mutual exclusion or other requirement not fulfilled by the application). Supplier shall document all conditions or timing requirements of any attributes if they are not always available.

11.5.5 The attribute value (all or a part of the list of ObjSetting) replied by the SetAttr service may be still unchanged if the acknowledge code of ObjStatus indicates a failure. In that case; the attribute value may be unchangeable, inhibited or blocked internally by any reason (e.g. corresponding hardware status, lack of privilege, mutual exclusion or other requirement not fulfilled by the application). The object providing the SetAttr service shall identify whether or not it is valid to change or set an attribute value. Supplier shall document all conditions and timing requirements of each attribute if they are not always allowed to be set.

11.5.6 *Object Specifier* — This parameter is used for specifying an owner object and may be omitted from the message. When included, it is a formatted text string that conforms to the description in Section 9. An object specifier is rejected by service providers that do not use owner or owned objects.

11.5.7 *Filter* — This is a optional list of qualifications to be applied to specified attribute and may be omitted from the message. When included, the complete filter is formed by a logical AND of each attribute qualification in the list.

11.5.8 *Requested Attribute* — The final parameter in the GetAttr service gives the names of one or more attributes of interest. These are the attributes whose values shall be returned in the response, in the order requested.

11.5.9 *Settings* — In the SetAttr service, a list of one or more attribute name/value pairs is specified. This is the set of desired attribute values. SetAttr rejects any attempt to set a read-only attribute and returns

information in Status, indicating the error and the attribute.

11.5.10 *ObjStatus* — This is a required parameter in the response message. It is a structure that consists of an acknowledge code and a list of status parameters. The acknowledge code is used to indicate errors that apply to the response as a whole. A status parameter is

a structure consisting of a code indicating a specific error and accompanying text providing additional information. Text is often used to inform a person of the results of an operation.

11.5.11 Tables 4 and 5 define the parameters for the GetAttr and SetAttr services, respectively.

Table 4 GetAttr Service

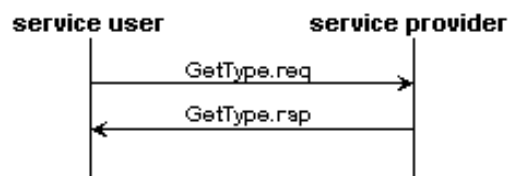
<i>Parameter</i>	<i>Req/Ind</i>	<i>Rsp/Conf</i>	<i>Description</i>
ObjSpec	C	-	The object specifier, used to specify the owner of the target object.
ObjType	M	-	The type of the target object.
(List of) ObjID	C	-	The identifier of the target object. If present, identifiers are treated as a pre-filter of the form “ObjID = <identifier>”, where the individual identifiers are joined with logical OR’s.
(List of) AttrFilter	C	-	An attribute filter to be applied to the target object. If both AttrFilter and ObjID are omitted, attributes are requested for all instances of the target object type.
(List of) AttrName	C	-	The name of a desired attribute. If the name is “ObjType,” and if ObjType (above) is omitted, then the object type of all subordinate objects is requested. If omitted, then all public attributes of the target object are requested.
(List of) ObjSetting	-	M	Attributes that match the specified characteristics, per qualifying object instance. Setting value may not be valid if corresponding ObjStatus shows a failure.
ObjStatus	-	M	Information concerning the success or failure of the request.

Table 5 SetAttr Service

<i>Parameter</i>	<i>Req/Ind</i>	<i>Rsp/Conf</i>	<i>Description</i>
ObjSpec	C	-	The object specifier, used to specify the owner of the target object.
ObjType	M	-	The type of the target object.
(List of) ObjID	C	-	The identifier of the target object. If present, is treated as a primary filter of the form “ObjID = <identifier>”.
(List of) AttrSetting	M	-	An attribute’s name and desired value.
(List of) ObjSetting	-	M	The attributes’ resulting values. Requested setting value(s) may have not changed if corresponding ObjStatus shows a failure.
ObjStatus	-	M	Information concerning the success or failure of the request.

11.6 *GetType* — GetType is a service that is directed at object types rather than object instances. It is used to ask an owner object for the object types that it owns.

11.6.1 Figure 9 shows the message flow for the GetType service.



**Figure 9
GetType Message Flow**

11.6.2 Comments for the ObjSpec and ObjStatus parameters of GetAttr and SetAttr also apply to the parameters for the GetType service.

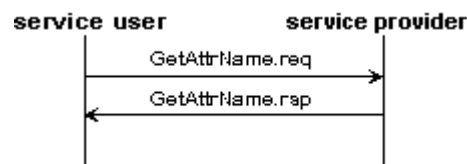
11.6.3 Table 6 defines the parameters for the GetType service.

Table 6 GetType Service

<i>Parameter</i>	<i>Req/Ind</i>	<i>Rsp/Conf</i>	<i>Description</i>
ObjSpec	C	-	The object specifier, used to specify the owner of the target object.
(List of)ObjType	-	M	The types of objects owned by the service provider.
ObjStatus	-	M	Information concerning the success or failure of the request.

11.7 *GetAttrName* — GetAttrName is a service that is directed at object types rather than object instances. It is used to ask for the names of the attributes of one or more objects owned by the service provider.

11.7.1 Figure 10 shows the message flow for the GetAttrName service.



**Figure 10
GetAttrName Message Flow**

11.7.2 Wild characters may be used in the object types requested.

11.7.3 Comments for the ObjSpec and ObjStatus parameters of GetAttr and SetAttr also apply to the parameters for the GetAttrName service.

11.7.4 Table 7 defines the parameters for the GetAttrName service.

Table 7 GetAttrName Service

<i>Parameter</i>	<i>Req/Ind</i>	<i>Rsp/Conf</i>	<i>Description</i>
ObjSpec	C	-	The object specifier, used to specify the owner of the target object.
(List of)ObjType	M	-	The type of the target object owned by the service provider. ObjType may use wild characters.
(List of)AttrName	-	M	The names of the attributes of each of the target object(s).
ObjStatus	-	M	Information concerning the success or failure of the request.

12 Additional Services

12.1 Additional services described in this section may have some restrictions when invoked. In some cases, services that are invoked may report a failure. For example: the owner of an object shall not delete the object during its lifecycle or while it is valid and required by another agent. Existence of an object may depend on preconditions, provided parameters, state of another object, corresponding operation, context of application, etc. Supplier shall document any restrictions or information regarding the validity of the additional services listed below.

12.1.1 This section defines the services required for additional operations defined in Section 10. These services are listed in Table 8.

Table 8 Additional Object Services

<i>Message Service Name</i>	<i>Type</i>	<i>Description</i>
Create	R	A request is made to create an object.
Delete	R	A request is made to delete an object.
Attach	R	A supervisor requests an object to attach itself to the supervisor.
AttachSetAttr	R	A supervisor requests to set one or more attributes of an attached object.
AttachSupervisedObject	R	A request is made to a supervisor to attach another object to the supervisor.
Detach	R	A supervisor requests an object to detach itself from the supervisor.
DetachSupervisedObject	R	A request is made to a supervisor to detach a specified attached object.
Reattach	R	A supervisor requests an object to reattach to itself to the new supervisor.
ObjectAction	R	A request for an object to perform an action.
ObjectActionCompletion	N	Notification that a delayed action has been completed, successfully or unsuccessfully.
GetServiceNames	R	A request to an object owner for a list of services supported by its owned objects.
GetServiceParameters	R	A request to an object for the service parameters supported for specified services.

12.2 *Object Services Parameter Dictionary* — Parameter definitions are provided in Section 11.1.

12.2.1 Table 9 defines the parameters used in services defined in Section 12, in addition to those parameters defined in Table 3.

Table 9 Additional Object Services Parameter Dictionary

<i>Parameter</i>	<i>Definition</i>	<i>Form</i>
AttachToken	Attach token provided by an object at the time it is attached or reattached.	Unsigned integer
TargetSpec	Object specifier of target object to attach or detach.	Formatted text for object specifier.

12.3 *Create* — The service user may request the service provider to create a new object and assign the value of one or more of its attributes. The request may be denied if an attempt is made to set attributes that are not allowed for that object type.

12.3.1 Parameters for Create are listed in Table 10.

Table 10 Create Service

<i>Parameter</i>	<i>Req/Ind</i>	<i>Rsp/Conf</i>	<i>Description</i>
ObjSpec	M	M	In the request, the object specifier of agent providing the object create service. In the response, the object specifier of the new instance. NOTE: The values may be different in the request and the response. The parameter in the request may be “ ” (an empty string) if there is no object or the owner object is outside of the domain of the receiving entity. If the entity is the equipment itself rather than a component of equipment, the owner may be defined by “Equipment” even if no such object is defined.
ObjType	M	-	The object type to be created.
(list of) AttrSetting	C	C	Initial attribute settings. Conditions for inclusion of an attribute are object-specific in both request and response.
ObjStatus	-	M	Information concerning the result of the requested operation.

12.4 *Delete* — The service user may request to delete an object.

12.4.1 Parameters for Delete are listed in Table 11.

Table 11 Delete Service

<i>Parameter</i>	<i>Req/Ind</i>	<i>Rsp/Conf</i>	<i>Description</i>
ObjSpec	M	-	The object specifier of the object to be deleted.
(list of) AttrSetting	C	C	Conditions for inclusion of an attribute are object-specific both request and response.
ObjStatus	-	M	Information concerning the result of the requested operation.

12.5 *Attach* — A service user may request an object to attach itself to the service user.

12.5.1 Parameters for Attach are listed in Table 12.

Table 12 Attach Service

<i>Parameter</i>	<i>Req/Ind</i>	<i>Rsp/Conf</i>	<i>Description</i>
ObjSpec	M	-	Specifier of server object to be attached.
AttachToken	-	M	Attach token. A value of zero shall be used if and only if the request to attach is denied.
(list of) AttrSetting	C	C	Settings for specific attributes. Object-dependent. Attributes in the list sent in the request and the response may be different.
ObjStatus	-	M	Information concerning the result of the requested operation.

12.6 *AttachSetAttr* — A supervisor may request an attached object to set one or more attributes at any time while the object is attached.

12.6.1 Parameters for AttachSetAttr are listed in Table 13.

Table 13 AttachSetAttr Service

<i>Parameter</i>	<i>Req/Ind</i>	<i>Rsp/Conf</i>	<i>Description</i>
ObjSpec	M	-	Specifier of server object to be attached.
AttachToken	M	-	Attach token.
(list of) AttrSetting	C	C	Settings for specific attributes. Object-dependent. Attributes in the list sent in the request and the response may be different.
ObjStatus	-	M	Information concerning the result of the requested operation.

12.7 *Detach* — The supervisor of an attached object may request the object to detach itself.

12.7.1 Parameters for Detach are listed in Table 14.

Table 14 Detach Service

<i>Parameter</i>	<i>Req/Ind</i>	<i>Rsp/Conf</i>	<i>Description</i>
ObjSpec	M	-	The specifier of the attached object.
AttachToken	M	-	Attach token.
(list of) AttrSetting	C	C	Attribute settings. Object-dependent. Attributes in the list sent in the request and the response may be different.
ObjStatus	-	M	Information concerning the result of the requested operation.

12.8 *Reattach* — An object may request an object attached to another supervisor to reattach to itself to the service user as its new supervisor.

12.8.1 Parameters for Reattach are listed in Table 15.

Table 15 Reattach Service

<i>Parameter</i>	<i>Req/Ind</i>	<i>Rsp/Conf</i>	<i>Description</i>
ObjSpec	M	-	Specifier of object to reattach.
AttachToken	-	M	Attach token. A value of zero shall be used if and only if the request to reattach is denied.
(list of) AttrSetting	C	C	Specific attribute settings. Object-dependent. Attributes in the list sent in the request and the response may be different.
ObjStatus	-	M	Information concerning the result of the requested operation.

12.9 *AttachSupervisedObject* — A service user may request a supervisor to attach a specified object.

12.9.1 Parameters for AttachSupervisedObject are listed in Table 16.

Table 16 AttachSupervisedObject Service

<i>Parameter</i>	<i>Req/Ind</i>	<i>Rsp/Conf</i>	<i>Description</i>
ObjSpec	M	-	The object specifier for the supervisor.
TargetSpec	M	-	The object specifier of the object to attach.
(list of) AttrSetting	C	C	Attribute settings. Object-dependent. May be omitted. Attributes in the list sent in the request and the response may be different.
ObjStatus	-	M	Information concerning the result of the requested operation.

12.10 *DetachSupervisedObject* — A service user may request a supervisor to detach a specified attached object.

12.10.1 Parameters for DetachSupervisedObject are listed in Table 17.

Table 17 DetachSupervisedObject Service

<i>Parameter</i>	<i>Req/Ind</i>	<i>Rsp/Conf</i>	<i>Description</i>
ObjSpec	M	-	The object specifier of the supervisor.
TargetSpec	M	-	The object specifier of the object to be detached.
(list of) AttrSetting	C	C	Attribute settings. Object-dependent. May be omitted. Attributes in the list sent in the request and the response may be different.
ObjStatus	reserved	reserved	Information concerning the result of the requested operation.

12.11 *ObjectAction* — A service user may request a specific action to be performed by, or on, a particular object. The object specifier represents either the target object as its owner, depending upon the object and action.

12.11.1 Completion, either normal (successful) or abnormal, may be indicated in either of two ways. If the action can be completed in a very short time, then it should be completed before a response to the request is sent. If the action takes more time than would be normal for the response, or if the action must wait for one or more conditions to be fulfilled, then a response to the request indicating the action will be performed later and the notification sent of its completion. See Section 12.12 below.

12.11.2 Figure 11 illustrates the first case, where the action is completed before the response is sent. The value of ObjectActionAcknowledgement should be set to any value other than “Action will be performed and notification sent later”.

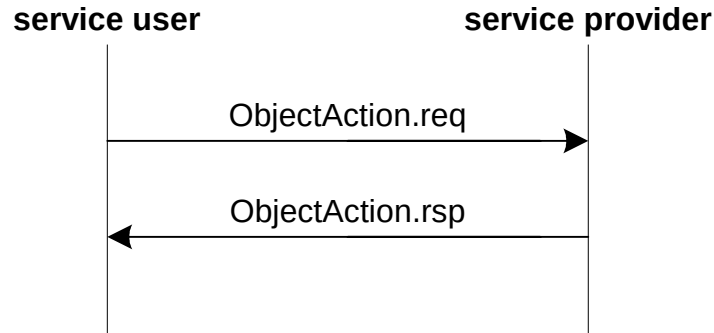


Figure 11
ObjectAction Request and Response

12.11.3 It is recommended that OperationID be always set to a unique non-zero value. However, if a value of zero is used, then the host should not send additional request until informed of the results of the outstanding request. Otherwise it may be ambiguous to which request an ObjectActionCompletion notification pertains. The definition of the target object type must include specification of the specific actions that it supports, together with the arguments that are required and the arguments that are optional for each action.

Table 18 ObjectAction Service

<i>Parameter</i>	<i>Req/Ind</i>	<i>Rsp/Conf</i>	<i>Description</i>
OperationID	M	M(=)	Operation identifier. Used where the service user may send multiple requests to link subsequent events to the original requests. May be zero otherwise.
ObjSpec	C	-	The object specifier of the target object or owner of the target object.
ObjectAction	M	-	Specific action requested.
(list of) ObjectActionParameter Request	C	-	A list of name/value pair arguments providing additional information about the request.
(list of) ObjectActionParameter Result	-	C	A list of name/value pair arguments providing additional information about the reply.
ObjectActionStatus	-	M	Information concerning the result of the requested action.
ObjectLinkID	-	M	Set to non-zero if and only if additional completion reports will be sent.

12.12 *ObjectActionCompletion Notification* — This notification is sent to the requestor of an earlier action when the completion of that action occurred after the response to the request was sent.

12.12.1 Figure 12 illustrates the scenario where the notification message is sent. The value of ObjectActionAcknowledgement should be set to “Action will be performed and notification sent later”.

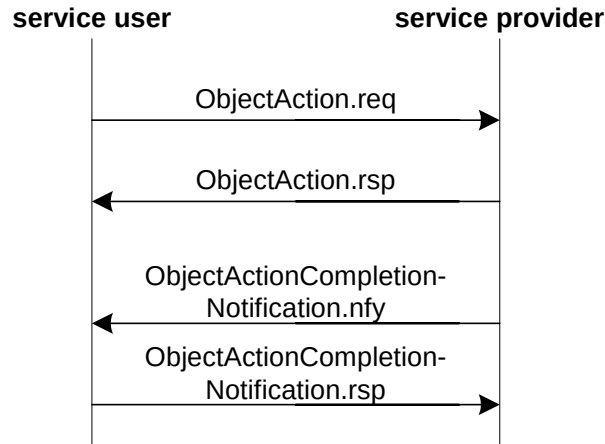


Figure 12
ObjectAction Request, Response, and Notification

Table 19 ObjectActionCompletion Notification

<i>Parameter</i>	<i>Req/Ind</i>	<i>Rsp/Conf</i>	<i>Description</i>
OperationID	M	-	Operation identifier. Set to the same value as in the original request.
OperationActionStatus	M	-	Information concerning the result of the service completion.
(list of) ObjectActionParameter Result	C	-	A list of name/value pair arguments providing additional information about the reply.
ObjectActionReportStatus	-	M	Indicates whether the report has been received with no errors.

12.13 *GetServiceNames* — A service user may ask an object owner for a list of the services supported by its owned objects.

12.13.1 The definition of the target object type must include specification of the specific actions that it supports, together with the arguments that are required and the arguments that are optional for each action.

Table 20 GetServiceNames Service

<i>Parameter</i>	<i>Req/Ind</i>	<i>Rsp/Conf</i>	<i>Description</i>
ObjSpec	C	-	The object specifier of the owner object.
(list of) ObjType	M	M	A list of object type values
ObjStatus	-	M	Information concerning the result of the requested action.
(list of) ObjServiceList	-	M	List of structures of the form ObjType, (list of) service names.

12.14 *GetServiceParameters* — This message may be sent to an object to determine the service parameters that are used by that object for a set of different services that it supports.

Table 21 GetServiceParameters

<i>Parameter</i>	<i>Req/Ind</i>	<i>Rsp/Conf</i>	<i>Description</i>
ObjSpec	C	-	The object specifier of the owner object.
ObjType	M	-	The type of object referenced.
(list of) ServiceName	M	-	A list of service names.
ObjStatus	-	M	Information concerning the result of the requested action.
(list of) ServiceParameterDef	-	C	List of structures that contain a service name with a list of parameter names.

13 Applications

13.1 This section provides examples of applications of object services.

13.2 *Get All of an Object's Attributes* — In the most straight-forward case, GetAttr is used to get all of the current attribute values of a single target object known to the provider of the GetAttr service. The object specifier is omitted, and the object type is set to the type of the target object. To get all of the object's attributes, both the attribute filter and the names of requested attributes are also omitted.

13.3 *Determine All Objects of a Specific Type and with Specific Characteristics* — To determine subordinate objects with specific characteristics, such as of type "Module" with an identifier *ObjID* starting with "ABC", set object type to "Module". In the attribute filter, set the attribute name to "ObjID", set the attribute value to "ABC*", and set the qualifying relationship to "is equal to" (or omit it).

13.4 *Determine Specific Attributes of a Specific Object Instance* — To determine the current value of certain attributes of a specific object, set the object type appropriately. Either set the list of *ObjID* to a list containing the one identifier of the target instance, or alternatively, in the attribute filter, set the attribute name to "ObjID", set the attribute value to the identifier, and set the qualifying relationship to "is equal to" (or omit it).

13.5 *Determine Types of Subordinate Objects* — To determine all object types that are owned by another object, the GetType service is used with the single parameter of the object specifier of the owner. The service provider returns an error of "Unknown object type" if it has no types of owned objects.

13.6 *Determine Names of Attributes of Subordinate Objects* — To determine the names of the attributes of specific objects owned by the service provider, the GetAttrName service is used with two parameters: the object specifier of the owner and a list of the types of the target objects. The service provided returns an error of "Unknown object type" if it has no types of owned objects.

14 Requirements for Compliance

14.1 Object Services are common to all service resources that define public objects with operations for getting (reading) and setting (writing) attribute values and for getting object types and attribute names for objects. Object services provide common definitions for public objects and object services that may be incorporated into these service resources, thereby avoiding unnecessary duplication.

14.2 *Fundamental Requirements* — All objects compliant to any part of OSS shall be subtypes of the top object. That is, they shall inherit (provide) the *ObjType* and *ObjID* attributes as described in Section 8.2.

14.2.1 They shall provide documentation of their public attributes in the form of an Object Attribute Definition table as described in Section 5.2.

14.2.2 They shall also provide the services GetAttr and SetAttr as defined in Section 11.5. Support for an owner's object specifier and filter are not required for fundamental compliance with OSS. If the service user provides parameters for scope or filter in its request, they may be ignored by the service provider. However, they shall not cause errors in the response due to their presence.

14.3 Additional Capabilities

14.3.1 *Filters* — Support for attribute filters in GetAttr and SetAttr requests is an optional capability.

14.3.2 *Owner Objects* — All owner objects (aggregates, containers, and supervisors) shall support both scope and filters for the GetAttr and SetAttr services, as defined in Sections 9.3, 11.2, and 11.3.

14.3.2.1 In addition, they shall provide the service GetType as defined in Section 11.6, with support for wild characters in the specification of object types.

14.3.3 *Multiple Inheritance Hierarchy* — Inheriting objects shall identify inherited objects with an inheritance expression in the object specifier.

APPENDIX 1

NOTICE: The material in this appendix is an official part of SEMI E39 and was approved by full letter ballot procedures on March 16 and April 21, 2000 by the Japanese Regional Standards Committee.

A1-1 Overview of Object Terminology

A1-1.1 This section provides an introduction to the basic terminology for object models.

A1-1.2 A *model* is an abstraction of a problem, a real-world phenomena, things, etc. for the purpose of understanding it. A model typically is a simplification that omits nonessential details. *Examples: architectural scale models, behavioral state models.*

A1-1.3 An *object* is an entity (concept, abstraction, or “real world” thing) with a particular behavior and with associated properties (information, or attributes). An *object type* (class) refers to a group of objects that have common (1) properties (but not specific values), (2) behavior, (3) relationships with other objects, and (4) semantics (public interfaces). The term “object” may be used either to refer to a type of object or to a particular instance of an object type.⁸ The notation used for diagrams of objects used in this document is described in Section A1-2.

A1-1.4 An *object model* is a static graphic model of objects to show structure — the identity of objects, their attributes and operations, and their relationships with one another.

A1-1.5 An *instance* of an object is an instantiation of an object type. For example, a specific installation of equipment is an instance of the object type “Equipment.”

A1-1.6 Objects have items of associated information called *attributes*. For example, for an object “Equipment,” attributes of interest include the manufacturer, model, serial number, and a logical user-assigned name (nickname). Attributes used to uniquely identify a particular instance of an object type is called an identifier. An object may have more than one *identifier*. Also, a set of more than one attribute may be used as an identifier. In the example of “Equipment,” name could be used as an identifier, and the combination of manufacturer and serial number also could be an identifier.

A1-1.7 Objects have *operations* that may be applied to or by an object type. Operations are functions or transformations that are either performed by, or on, an object. Operations of interest to Object Services are “get (read) attributes,” “set (write) attributes,” and “get

type.” A *service* provides a *service user* with an interface to the functionality of the operation.

A1-1.8 A high-level model may show a particular type of object with certain operations, while a more detailed model shows that a second type of object actually performs one or more of the operations on the first object. As an example, a model might show a process program object with the operation “delete,” to show that the “delete” operation is inherently associated with the process program. A more detailed model might show that a “process program manager” actually performs the operation of deleting a process program.

A1-1.9 A *method* is an implementation of an operation (e.g., the software code that performs the operation).

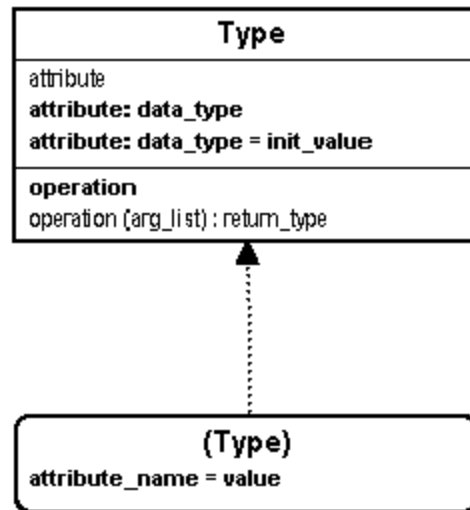


Figure A1-1
Object Type and Instance

⁸ The term “object” is used by some authors to only refer to instances and by other authors as a synonym for object type.

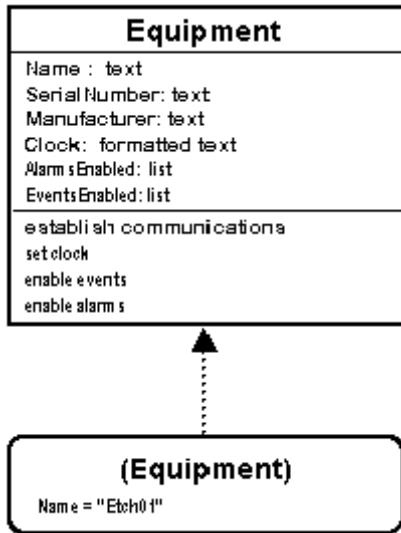


Figure A1-2
Example of Object Type and Instance

A1-2 Object Modeling Technique (OMT) Notation

A1-2.1 The material contained in this section is provided as a reference for auxiliary information.

A1-2.1.1 Object Modeling Technique (OMT) is a graphical notation for models of objects that is useful for analyzing a wide variety of problems, in all phases of software design and development, and in preparing documentation. OMT was developed by Rumbaugh, Blaha, et al., in *Object Modeling and Design*, Prentice Hall, Englewood Cliffs, NJ, ©1991. OMT has been adopted by different standards to describe requirements in terms of objects and relationships between objects. The purpose of this document is to provide a description of OMT notation as a reference for such SEMI standards and other SEMI documents.

A1-2.2 *Basic Notation* — Figure A1-1 illustrates notation for object type, showing object type, object attributes, object operations, object instance, and the relationship between an object type and instance. Figure A1-2 provides an example.

A1-2.2.1 Object types in OMT are shown as rectangles. The rectangle may be further subdivided into two or three parts. The object type always appears at the top. Object *attributes* are shown in a second part of the rectangle. *Operations* are shown in a third part of the rectangle. High-level models often omit the operations section and sometimes omit the attributes section.

A1-2.2.2 The *name* of an attribute may be followed by additional details, such as data type and initial value.

A1-2.2.3 *Instances* of objects are shown as rounded rectangles. The *instantiation* relationship is shown by a dotted arrow from an instance of an object type to the type.

A1-2.2.4 The example for an object type “Equipment” is shown in Figure A1-2; here “Etch01” is a specific real-world installation of type “Equipment.”

A1-2.3 *Associations* — An association describes a set of potential bi-directional relationships between instances of objects. An association uses a formal structure and semantics.

A1-2.3.1 An association shows a specific multiplicity at both ends of the relationship. For example, a single factory manufacturing system may be associated with many equipment (one-to-many). Possible multiplicities of association types are shown in Figure A1-3.

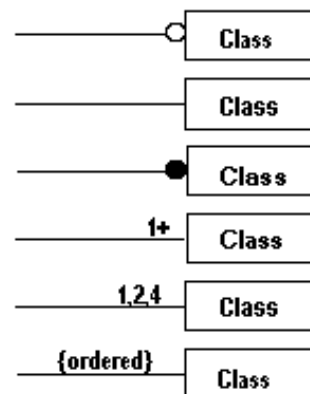


Figure A1-3
Multiplicities of Associations

A1-2.3.2 Figure A1-4 shows an example of a one-to-one relationship. The name of the association (using a verb) appears above the line connecting a pair of objects and describes the relationship that the object to the left (or above) has to the object to the right (or below).

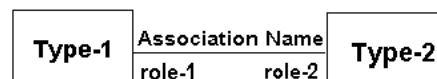


Figure A1-4
Association of Objects

A1-2.3.3 The name of the role of each object within the relationship may be placed beneath the line and near that object. Role names are nouns. In the example above, the roles might be “supervisor” and “subordinate.” Role names are optional.

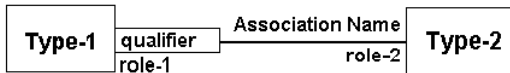


Figure A1-5
Qualified Association

A1-2.3.4 A *qualified association*, shown in Figure A1-5, uses a *qualifier* to reference an object. *Qualifiers* may be shown to specify the identifier used by one object to associate with instances of the other object. If Type-1 object is “Factory Host” and Type-2 object is “Equipment,” a user-assigned “name” attribute may be used as a qualifier for the equipment.

A1-2.3.5 Figure A1-6 illustrates a *link object*, where link attributes are placed in an object box that is attached to the association with a loop. A *link object* may be used to show attributes that are dependent on the association between instances of one object type to instances of another type.

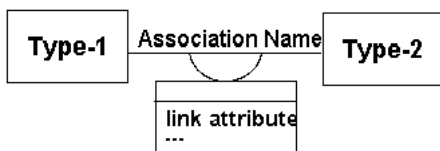


Figure A1-6
Link Object

A1-2.3.6 Figure A1-7 gives an example of a link object for the association “Authorized On” between an “Authorized User” and “Equipment.”

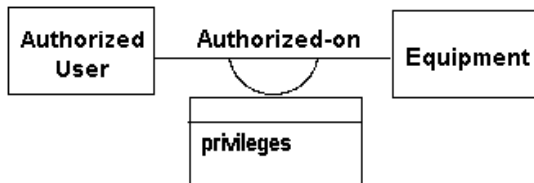


Figure A1-7
Example of a Link Object

A1-3 Generalization and Inheritance

A1-3.1 Figure A1-8 shows the concept of *generalization*. Generalization categorizes a set of object types and allows the abstraction of their common features into a supertype, with refinements of the *supertype* shown as *subtypes*. The subtype may be thought of as a specialization of the supertype. For

example, “vehicle” is a supertype with many subtypes, including “aircraft,” “automobile,” “cart,” and “robot.”

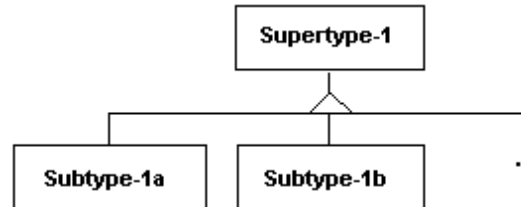


Figure A1-8
Non-Overlapping Subtypes

A1-3.1.1 An instance of a subtype is also an instance of its supertype. Attributes and operations of the supertype are inherited by all the subtypes. That is, the subtype has all of the same attributes and operations of the supertype, and in addition it adds attributes and operations of its own.

A1-3.1.2 The OMT notation for the generalization (supertype/subtype) relationship is shown by a solid triangle. White triangles indicate that the subtypes are *non-overlapping*; an instance of one subtype may not be an instance of another subtype.

A1-3.1.3 The method for a subtype of an object might be different from that of the supertype or from a different subtype, perhaps due to more specialized knowledge that allowed a more efficient implementation for the subtype. As an example, a Geometrical Figure object type might have the operation “draw.” The methods for subtypes of “circle” and “rectangle” would differ, however.

A1-3.1.4 Figure A1-10 gives an example of an “agent” as a generalization of various types of active entities that might be found in a factory, and “equipment” as a generalization of specific kinds of equipment.

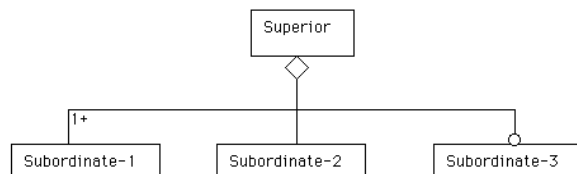


Figure A1-9
Aggregation with Two Component Types

A1-3.2 *Object Composition and Containment* — Figures A1-9 and A1-10 illustrate the diamond notation used to show aggregation. An aggregation object is also referred to as an assembly in object-oriented literature.

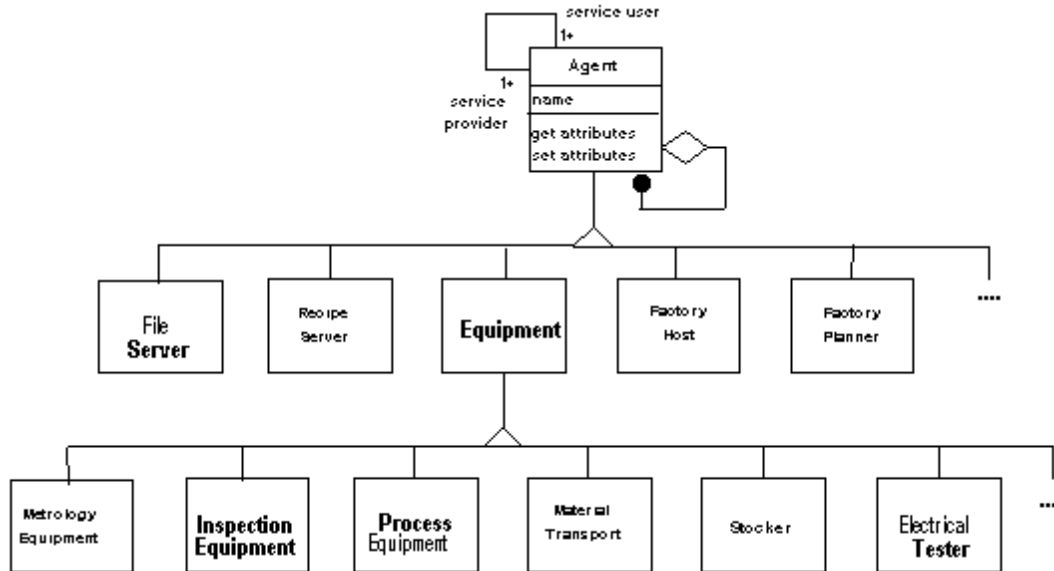


Figure A1-10
An Example of Generalization

A1-3.2.1 An aggregation object is composed of other objects called the *components* of the aggregation. Components may be of the same type as the aggregation (illustrated in Figure A1-10), or they may be of one or more different types.

A1-3.2.2 In Figure A1-9, the “Superior” object type is composed of one or more objects of type “Subordinate-1,” of exactly one component of object type “Subordinate-2,” and of zero or one objects of type “Subordinate-3.”

A1-3.2.3 In addition, an aggregate also may be optionally composed of other objects of the same type, as Figure A1-10 illustrates. The “agent” in this figure is a very general supertype of a factory system. Additional subtypes of agents, not shown in the figure, could include cells, clusters, and cluster modules. Cells, with equipment as components, and clusters, with modules as components, are examples of types of agents that are aggregates composed of other subtypes of agent.

NOTICE: SEMI makes no warranties or representations as to the suitability of the standard set forth herein for any particular application. The determination of the suitability of the standard is solely the responsibility of the user. Users are cautioned to refer to manufacturer’s instructions, product labels, product data sheets, and other relevant literature respecting any materials mentioned herein. These standards are subject to change without notice.

The user’s attention is called to the possibility that compliance with this standard may require use of copyrighted material or of an invention covered by patent rights. By publication of this standard, SEMI takes no position respecting the validity of any patent rights or copyrights asserted in connection with any item mentioned in this standard. Users of this standard are expressly advised that determination of any such patent rights or copyrights, and the risk of infringement of such rights, are entirely their own responsibility.



SEMI E39.1-0703

SECS-II PROTOCOL FOR OBJECT SERVICES STANDARD (OSS)

This standard was technically approved by the Global Information & Control Committee and is the direct responsibility of the North American Information & Control Committee. Current edition approved by the North American Regional Standards Committee on March 12, 2003. Initially available at www.semi.org May 2003; to be published July 2003. Originally published in 1995; previously published November 2002.

1 Purpose

1.1 This document maps the services and data of the parent document to SECS-II streams and functions and data definitions.

2 Scope

2.1 This document applies to all implementations of Object Services that use the SECS-II message protocol (SEMI E5).

NOTICE: This standard does not purport to address safety issues, if any, associated with its use. It is the responsibility of the users of this standard to establish appropriate safety and health practices and determine the applicability of regulatory or other limitations prior to use.

3 Referenced Documents

SEMI E5 — SEMI Equipment Communications Standard 2 Message Content (SECS-II)

SEMI E39 — Object Services Standard: Concepts, Behavior, and Services

NOTICE: Unless otherwise indicated, all documents cited shall be the latest published versions.

4 Mapping of Object Services

4.1 Table 1 shows the specific SECS-II streams and functions that shall be used for SECS-II implementations for the object services defined in OSS.

Table 1 Services Mapping Table

<i>Message Name</i>	<i>Stream, Function</i>	<i>SECS-II Name</i>
GetAttr.req	S14,F1	GetAttr Request
GetAttr.rsp	S14,F2	GetAttr Data
SetAttr.req	S14,F3	SetAttr Request
SetAttr.rsp	S14,F4	SetAttr Data
GetType.req	S14,F5	GetType Request
GetType.rsp	S14,F6	GetType Data
GetAttrName.req	S14,F7	GetAttrName Request
GetAttrName.rsp	S14,F8	GetAttrName Data
Create.req	S14,F9	Create Object Request
Create.rsp	S14,F10	Create Object Acknowledge
Delete.req	S14,F11	Delete Object Request
Delete.rsp	S14,F12	Delete Object Acknowledge
Attach.req	S14,F13	Object Attach Request
Attach.rsp	S14,F14	Object Attach Acknowledge
AttachSetAttr.req	S14,F15	Attached Object Action Request
AttachSetAttr.rsp	S14,F16	Attached Object Action Acknowledge
AttachSupervisedObject.req	S14,F17	Supervised Object Action Request
AttachSupervisedObject.rsp	S14,F18	Supervised Object Action Acknowledge
Detach.req	S14,F15	Attached Object Action Request
Detach.rsp	S14,F16	Attached Object Action Acknowledge
DetachSupervisedObject.req	S14,F17	Supervised Object Action Request
DetachSupervisedObject.rsp	S14,F18	Supervised Object Action Acknowledge
Reattach.req	S14,F13	Object Attach Request

<i>Message Name</i>	<i>Stream, Function</i>	<i>SECS-II Name</i>
Reattach.rsp	S14,F14	Object Attach Acknowledge
GetServiceNames.req	S14,F25	Get Service Name Request
GetServiceNames.rsp	S14,F26	Get Service Name Data
GetServiceParameters.req	S14,F27	Get Service Parameter Name Request
GetServiceParameters.rsp	S14,F28	Get Service Parameter Name Data
ObjectAction.req	S14,F19	Generic Service Request
ObjectAction.rsp	S14,F20	Generic Service Acknowledge
ObjectActionCompletion.nfy	S14,F21	Generic Service Completion Information
ObjectActionCompletion.rsp	S14,F22	Generic Service Completion Acknowledge

5 Service Parameter Mapping

5.1 Table 2 shows the mapping between message parameters defined by OSS and data items defined by SECS-II. For parameters specified in the definitions of an OSS service, either the parameters themselves, or individual elements of complex parameters, map to a specific data item.

Table 2 Service Parameters Item Mapping Table

<i>Parameter Name</i>	<i>SECS-II Data Item</i>
AttachToken	OBJTOKEN
AttrData	ATTRVAL
AttrName	ATTRID
AttrReIn	ATTRRELN
ErrCode	ERRCODE
ErrText	ERRTEXT
ObjAck	OBJACK
ObjectAction	SVCNAME
ObjectActionAcknowledgement	SVCACK
ObjectActionParameterRequest	L,2 1. <SPNAME> 2. <SPVAL>
ObjectActionParameterResult	L,2 1. <SPNAME> 2. <SPVAL>
ObjectActionReportStatus	DATAACK
ObjectActionStatus	L,2 1. <SVCACK> 2. Status
ObjID	OBJID
ObjSpec	OBJSPEC
ObjType	OBJTYPE
OperationID	OPID
ServiceParameterName	SPNAME
ServiceParameterValue	SPVAL

Parameter Name	SECS-II Data Item
Status	L,n 1. L,2 1.<ERRCODE ₁ > 2.<ERRTEXT ₁ > : n. L,2 1. <ERRCODE _n > 2. <ERRTEXT _n >
TargetSpec	TARGETSPEC

5.2 SECS-II Data Items Without Corresponding SEMI E94 Parameters

5.2.1 Table 3 contains the SECS-II data items that do not correspond to SEMI E39's service parameters.

Table 3 Additional Data Item Requirements Table

Function	SECS-II Data Item
Used to satisfy SECS-II conventions for linking a multi-block inquiry with a subsequent multi-block message. Neither required nor specified by OSS.	DATAID
Used to inform receiver of total message length size for SECS-II multi-block conventions. Neither required nor specified by SEMI E39.	DATALLENGTH
Used to satisfy SECS-II multi-block requirements. Neither required nor specified by OSS.	GRANT

6 Data Item Format Restrictions

ATTRID Format: 20

The ASCII version of ATTRID is restricted to the characters from 20₁₆ through 7E₁₆, excluding the following characters: the “greater than” symbol “>”, the colon “:”, the question mark “?”, the asterisk “*”, and the tilde “~”. The space character (20₁₆) may not be used as the first or last character. Maximum length is 40 characters.

OBJTYPE Format: 20

The ASCII version of OBJTYPE is restricted to the characters from 20₁₆ through 7E₁₆, excluding the following characters: the “greater than” symbol “>”, the colon “:”, the question mark “?”, the asterisk “*”, and the tilde “~”. The space character (20₁₆) may not be used as the first or last character. Maximum length is 40 characters.

OBJID Format: 20

The ASCII version of OBJID is restricted to the characters from 20₁₆ through 7E₁₆, excluding the following characters: the “greater than” symbol “>”, the colon “:”, the question mark “?”, the asterisk “*”, and the tilde “~”. The space character (20₁₆) may not be used as the first or last character. The equipment shall allow the OBJID to be any length from 1 to 80 characters, inclusive. A zero length OBJID implies that no OBJID is specified.

NOTICE: SEMI makes no warranties or representations as to the suitability of the standards set forth herein for any particular application. The determination of the suitability of the standard is solely the responsibility of the user. Users are cautioned to refer to manufacturer's instructions, product labels, product data sheets, and other relevant literature respecting any materials mentioned herein. These standards are subject to change without notice.

The user's attention is called to the possibility that compliance with this standard may require use of copyrighted material or of an invention covered by patent rights. By publication of this standard, SEMI takes no position respecting the validity of any patent rights or copyrights asserted in connection with any item mentioned in this standard. Users of this standard are expressly advised that determination of any such patent rights or copyrights, and the risk of infringement of such rights, are entirely their own responsibility.